Vedi Collections — Admin Panel

Corrected flow & build plan · Manual catalog management (role-based) · for Aastha

You and your client agreed to drop the WhatsApp catalog API and let her manage products by hand through an admin panel on the site. Your notebook flow was solid — the *idea* is right. The fixes below are mostly about **how** a few of those steps should actually be built so they're safe and don't break later. Each point is written as: what you had → why it's a problem → what to do instead.

The 6 things to change

1 "Admin = email in a list" → store a role in the database

WHAT YOU HAD

A fixed list of admin emails (vedicollections.official, your emails, etc.). To check "is this person an admin?" you'd check if their email is in that list.

WHY IT'S A PROBLEM

The list lives *inside your code*. Every time you want to add or remove an admin, you'd have to edit the code and redeploy the whole website. It also mixes up two different questions that should stay separate: **"who are you?"** (login) and **"what are you allowed to do?"** (permission).

WHAT TO DO INSTEAD

Add one column to your users table called `role`, which is either `customer` or `admin`. When someone logs in, you read their role from the database. To make someone an admin, you change one value in the database — no code change, no redeploy. Set your handful of admins to `admin` once at the start.

2 Hiding the "Admin" tab is not security

WHAT YOU HAD

An "Admin" tab that only shows in the navbar for admins.

WHY IT'S A PROBLEM

Hiding a button on the website doesn't actually stop anyone. The website (frontend) is just a convenient door — but your API is the real entrance, and a person can knock on it directly with a tool like Postman, completely skipping your website. So hiding the tab is like hiding a door while leaving it unlocked.

WHAT TO DO INSTEAD

Every admin action on the **server** must check the role before doing anything. In FastAPI you write this check once as a reusable dependency and attach it to every admin route:

```
# one check, reused on every admin endpoint
def require_admin(user = Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(403, "Admins only")
    return user
```

Hiding the tab is still nice for tidiness — just keep it as a convenience, never as the protection.

3 Use the right HTTP method for each action

WHAT YOU HAD

"update → PATCH/POST" and "POST → new launch."

WHY IT'S A PROBLEM

"PATCH/POST" for update is mixing two things. The methods each have a clear job, and keeping them clean makes your API predictable.

WHAT TO DO INSTEAD — SIMPLE RULE:

Method	Means	Your use
`POST`	Make something new	New product launch ✓ (you had this right)
`PATCH`	Change part of it	Update price / stock count of a product
`PUT`	Replace the whole thing	Rarely needed — skip unless you want a full edit
`DELETE`	Remove it	Remove a product — but see point 4 first

4 Don't actually delete products — "soft delete" them

WHAT YOU HAD

"Removed item from stock → occur change directly" (i.e. delete the product).

WHY IT'S A PROBLEM

Old customer orders point to that product. If you truly delete the product's row, those past orders now point to nothing — broken order history, broken receipts, possible crashes when someone views an old order.

WHAT TO DO INSTEAD

Add a column like `is_active` (true/false). To "remove" a product, set `is_active = false`. It disappears from the shop, but the record still exists so old orders stay intact — and you can

bring it back any time. This is called a **soft delete**. So your `DELETE` endpoint just flips this flag; it doesn't drop the row.

5 Emails: save first, send in the background — and keep a log table

WHAT YOU HAD

On any change → send a confirmation email + send a list of all changes to all admin emails.

WHY IT'S A PROBLEM

Two issues. First, if you send the email *during* the request, the admin has to wait for the email to finish, and if the email service is down, your whole "save change" could fail along with it. The email should never block the actual change. Second, email by itself is a weak record — emails get deleted, buried, or lost. It's a bad place to keep your official history.

WHAT TO DO INSTEAD

1) Save the change to the database **first**, return "saved" to the admin instantly, **then** send the emails in the background. FastAPI has `BackgroundTasks` built in exactly for this.

2) Create a `change_log` table that records every change (who did it, what changed, old value → new value, when). **That** is your real history. The emails just become friendly notifications sitting on top of it.

6 The "go live after 24h / at a chosen time" part needs a background worker

WHAT YOU HAD

Changes go live: immediately, OR at a chosen date/time, OR after 24 hours.

WHY IT'S A PROBLEM

A single API request finishes in milliseconds. It cannot "wait 24 hours and then do something" — once the request is done, it's gone. Something separate has to act later, at the scheduled moment.

WHAT TO DO INSTEAD

Give each product two fields: a `status` (`draft` / `scheduled` / `live` / `inactive`) and a `go_live_at` time.

- **Immediate change** (e.g. removing an item): just do it now — set status straight to `live` or `inactive`.
- **Scheduled launch**: set status = `scheduled`, `go_live_at` = the chosen time (or now + 24h).

Then a small background job runs every minute, asks "any scheduled items whose `go_live_at` time has passed?", and flips those to `live`. Tools for this in your stack:

APScheduler (easiest to start, runs inside FastAPI), or a system cron job, or a task queue like Celery later if it grows.

Plus — the "Add rate limit" note at the top of your page. Good instinct. Put rate limiting especially on your **login** route (stops password-guessing) and your **admin write** routes. In FastAPI the simplest library is `slowapi` — e.g. allow only N login attempts per minute per IP.

Your corrected flow, rewritten cleanly

Login → read the user's **role** from the database.

→ **customer**: browse & buy only. Sees products that are `live` and `active`.

→ **admin**: sees the Admin tab. Every admin action below is protected on the server by `require_admin`.

- **New launch** → `POST` → choose: live now, OR schedule (pick date/time, else now + 24h) → status set accordingly.
- **Update** stock / price → `PATCH`.
- **Remove** item → `DELETE` → soft delete (`is_active = false`), happens immediately.

→ On **any** admin change: write a row to `change_log`, then (in background) email a confirmation + the change to all admins.

→ A background job flips `scheduled` products to `live` when their time arrives.

Database tables you'll need

users

Column	What it holds
`id`	Primary key
`name`, `email`	Basic identity (email unique)
`password_hash`	Hashed password (bcrypt — you already do this)
`role`	**NEW.** `customer` or `admin`
`created_at`	Timestamp

products

Column	What it holds

`id`	Primary key
`name`, `description`, `price`	Product details
`stock_quantity`	How many in stock
`images`	Image URLs (list / separate table later)
`status`	**NEW.** draft / scheduled / live / inactive
`is_active`	**NEW.** true/false — soft delete flag
`go_live_at`	**NEW.** when a scheduled product becomes live
`created_at`, `updated_at`	Timestamps

change_log

Column	What it holds
`id`	Primary key
`admin_id`	Who made the change (→ users.id)
`product_id`	Which product
`action`	create / update / remove / launch
`field_changed`	e.g. "price", "stock_quantity"
`old_value`, `new_value`	Before → after
`created_at`	When it happened

Endpoint map

Endpoint	Who	Does
`GET` /products	Public	List products that are live + active only
`GET` /products/{id}	Public	One product (if live + active)
`POST` /admin/products	Admin	New launch (now or scheduled)
`PATCH` /admin/products/{id}	Admin	Update price / stock / details
`DELETE` /admin/products/{id}	Admin	Soft delete (is_active = false)
`GET` /admin/products	Admin	See everything incl. draft / scheduled / inactive
`GET` /admin/change-log	Admin	Full change history

Suggested build order

Lean and in order — each step works on its own, so you ship something usable early instead of waiting until the whole thing is built.

- [ ] **1. Role + access.** Add the `role` column, set your admins, write the `require_admin` dependency. Test that a customer gets blocked from an admin route.
- [ ] **2. Product CRUD.** POST / PATCH / GET with the soft-delete DELETE. Public GET filters to live + active.
- [ ] **3. Change log.** Add the table; write a row on every admin change.
- [ ] **4. Background emails.** Move email sending into `BackgroundTasks` after the save.
- [ ] **5. Scheduling.** Add `status` + `go_live_at`; add the APScheduler job that flips scheduled → live.
- [ ] **6. Rate limit.** Add `slowapi` on login + admin write routes.

One honest note: steps 1–2 already give you a working admin panel. Don't let 3–6 turn into a reason to delay shipping a usable version to your client. Build 1 and 2, show her, then layer the rest on.